



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

Образовательная программа 15.03.06
«Мехатроника и робототехника»

Дисциплина «Программирование на C++»

Отчет
По заданию на стажировку

Принял:

подпись

Гаврилов А.Г.

Выполнили
студенты:

АДБ-24-09

группа

подпись

Курдюков М.М.

подпись

Королев Е.Е.

подпись

Симаков Ф.И.

подпись

Спивак Н.С.

подпись

Черных С.В.

Москва 2026

1. Введение

Файловый менеджер является программой с простым интерфейсом для работы с файлами и директориями. В данном приложении интерфейс реализован в консоли: пользователь видит список элементов текущей папки и управляет курсором с помощью клавиатуры.

При запуске программа получает путь к текущей рабочей директории, считывает ее содержимое и выводит список файлов и папок. Первый элемент списка автоматически становится выбранным. Далее программа работает в цикле: сначала перерисовывает экран, затем считывает нажатую клавишу и выполняет соответствующее действие. Приложение ориентировано на выполнение базовых операций файлового менеджера.

2. Особенности реализации

Для доступа к файловой системе используется стандартная библиотека `std::filesystem`, что позволяет получать список файлов, определять тип элемента, удалять, копировать и перемещать объекты файловой системы.

Основная особенность реализации заключается в том, что приложение хранит текущую директорию, список найденных элементов и позицию курсора в глобальных переменных. Для обработки клавиш используется функция `_getch()` из `conio.h`, поэтому приложение рассчитано на консоль Windows.

Для корректного вывода русских имен файлов и просмотра русскоязычного текста используется широкая строка `wstring` и настройка режима вывода консоли через `_setmode`. При чтении файла текст сначала загружается как набор байтов, после чего функция преобразования пытается интерпретировать его как UTF-8, а при ошибке использует кодировку Windows-1251.

2.1. Используемые библиотеки

`iostream` — вывод данных в консоль через `wcout`.

`vector` — хранение списка файлов и директорий текущей рабочей папки.

`string` — хранение текста файла перед выводом на экран.

`filesystem` — работа с путями, проверка типа элемента, копирование, удаление и перемещение файлов.

`fstream` — чтение содержимого файла в режиме просмотра.

`conio.h` — получение нажатий клавиш без необходимости нажимать Enter.

`windows.h` — использование функций Windows API для преобразования кодировок.

`io.h` и `fcntl.h` — настройка режима вывода консоли для поддержки широких символов.

2.2. Структура данных `Item`

```
struct Item { fs::path path; bool isDir; };
```

Структура `Item` описывает один элемент файловой системы, который отображается в списке файлового менеджера. Поле `path` хранит полный путь к файлу или директории. Поле `isDir` показывает тип элемента: если значение `true`, элемент является директорией, если `false` — файлом.

2.3. Глобальные переменные состояния

```
fs::path currentDir = fs::current_path();
```

Переменная `currentDir` хранит путь к текущей рабочей директории. При запуске она получает значение директории, из которой была запущена программа.

```
vector<Item> items;
```

Контейнер `items` хранит все элементы текущей директории. После перехода в другую папку или после удаления, копирования и перемещения этот список обновляется функцией `load()`.

```
int cursor = 0;
```

Переменная `cursor` хранит индекс выбранного элемента. Именно этот элемент считается активным, и к нему применяются операции открытия, удаления, отметки или просмотра.

```
int topIndex = 0;
```

Переменная `topIndex` нужна для прокрутки списка. На экран выводятся не все элементы сразу, а только ограниченное количество строк. `topIndex` показывает индекс первого видимого элемента.

```
fs::path marked; bool hasMarked = false;
```

Переменная `marked` хранит путь к отмеченному файлу или папке. Флаг `hasMarked` показывает, существует ли сейчас отмеченный элемент. Отметка сохраняется даже при переходе в другую директорию, что позволяет скопировать или переместить выбранный ранее объект в новую рабочую папку.

2.4. Подробное описание функций

Функция `toWide()`

```
wstring toWide(const string& s)
```

Она преобразует обычную строку `string` в широкую строку `wstring`, чтобы текст можно было вывести через `wcout` в консоль Windows.

Сначала функция предполагает, что входная строка записана в кодировке UTF-8, поэтому в переменную `sr` записывается значение `CP_UTF8`. Затем вызывается `MultiByteToWideChar()` с флагом `MB_ERR_INVALID_CHARS`. Этот вызов не выполняет само преобразование, а определяет количество широких символов, которое потребуется для результата. Если размер результата равен нулю, значит строка не была корректной UTF-8. В этом случае функция переключается на кодировку Windows-1251, которая часто используется для русского текста в Windows. После определения нужной кодировки создается строка `wstring` нужного размера. Второй вызов `MultiByteToWideChar()` уже записывает преобразованные символы в созданную строку `wstring`. Готовая широкая строка возвращается вызывающему коду.

Функция возвращает текст в формате `wstring`, пригодный для вывода русских и английских символов через `wcout`.

Функция `load()`

```
void load()
```

Она загружает содержимое текущей рабочей директории в список `items`.

Сначала вызывается `items.clear()`, чтобы удалить старый список файлов и папок. Далее запускается цикл `for` по объекту `fs::directory_iterator(currentDir)`. Этот итератор перебирает все элементы, находящиеся в текущей директории. Для каждого найденного элемента создается объект `Item`. В поле `path` записывается полный путь `p.path()`. В поле `isDir` записывается результат `fs::is_directory(p.path())`, то есть проверка, является ли найденный элемент

директорией. Созданный объект добавляется в вектор `items` с помощью `push_back()`. После загрузки списка курсор сбрасывается на первый элемент: `cursor = 0`. Индекс верхней видимой строки также сбрасывается: `topIndex = 0`. Это нужно, чтобы после обновления директории список показывался с начала.

Вектор `items` содержит актуальный список файлов и папок текущей директории, а курсор установлен на начало списка.

Функция `draw()`

```
void draw()
```

Она перерисовывает консольный интерфейс файлового менеджера.

Сначала вызывается `system("cls")`, чтобы очистить консоль. Затем выводится заголовок `FILE MANAGER` и путь текущей рабочей директории `currentDir`. После этого программа проверяет `hasMarked`. Если элемент отмечен, выводится путь `marked`. Если отметки нет, выводится строка `Marked: none`. Переменная `visible` задает количество строк списка, которые одновременно видны на экране. Здесь `visible` равна 10. Если `cursor` стал меньше `topIndex`, значит курсор поднялся выше видимой области. Тогда `topIndex` сдвигается к текущему положению курсора. Если `cursor` стал больше или равен `topIndex + visible`, значит курсор ушел ниже видимой области. Тогда `topIndex` пересчитывается так, чтобы выбранный элемент снова попал на экран. Переменная `last` определяет индекс конца видимой части списка. Используется `min()`, чтобы не выйти за пределы вектора `items`. В цикле от `topIndex` до `last` функция выводит элементы списка. Перед выбранным элементом печатается символ `>`, перед отмеченным элементом печатается символ `*`. Для папок выводится метка `[D]`, для файлов — `[F]`. После метки выводится имя файла или папки без полного пути. Внизу экрана выводится подсказка по клавишам управления.

На экране появляется актуальное состояние файлового менеджера: текущая папка, отмеченный элемент, список файлов и подсказки управления.

Функция `key()`

```
int key()
```

Она считывает нажатие клавиши и переводит его во внутренний числовой код команды.

Функция вызывает `_getch()`, чтобы получить код нажатой клавиши. В отличие от обычного ввода через `cin`, `_getch()` не требует нажатия Enter. Некоторые клавиши, например стрелки и функциональные клавиши, возвращают сначала служебный код 0 или 224. Поэтому при таком значении функция вызывает `_getch()` повторно. Если второй код равен 72, возвращается команда 1 — движение вверх и т.д. Для неизвестных клавиш возвращается 0. В основном цикле такой код просто не приводит к действию.

Функция возвращает число от 1 до 10, соответствующее команде пользователя, или 0 для неподдерживаемой клавиши.

Функция `openDir()`

```
void openDir()
```

Она открывает выбранную директорию и делает ее текущей рабочей папкой.

Сначала функция проверяет, пуст ли список `items`. Если список пустой, открывать нечего, поэтому выполнение сразу завершается. Затем проверяется поле `isDir` у элемента `items[cursor]`. Если выбранный элемент не является директорией, функция ничего не делает. Если выбранный элемент является директорией, путь `currentDir` заменяется на путь выбранной папки. После изменения `currentDir` вызывается `load()`, чтобы считать содержимое новой директории и обновить список элементов.

Если курсор стоял на папке, пользователь переходит внутрь этой папки, а список на экране обновляется.

Функция `goParent()`

```
void goParent()
```

Она переводит файловый менеджер из текущей директории в родительскую директорию.

Функция получает родительский путь с помощью `currentDir.parent_path()`. Полученное значение записывается обратно в `currentDir`. После изменения текущего пути вызывается `load()`, чтобы загрузить файлы и папки родительской директории. Курсор и прокрутка автоматически сбрасываются функцией `load()`.

Пользователь возвращается на один уровень выше в файловой системе.

Функция `del()`

```
void del()
```

Она удаляет выбранный файл или выбранную директорию.

Сначала проверяется, пуст ли список `items`. Если элементов нет, удалять нечего. Для выбранного элемента берется путь `items[cursor].path`. Вызов `fs::remove_all()` удаляет объект по этому пути. Если это файл, удаляется файл. Если это директория, удаляется директория вместе со всем содержимым. После удаления вызывается `load()`, чтобы обновить список элементов текущей папки. Поскольку `load()` сбрасывает `cursor` в 0, после удаления выделение возвращается к первому элементу списка.

Выбранный элемент удаляется из файловой системы, а экран после следующей перерисовки показывает обновленный список.

Функция `mark()`

```
void mark()
```

Она запоминает выбранный файл или выбранную директорию для дальнейшего копирования или перемещения.

Сначала проверяется, пуст ли список `items`. Если список пустой, отметить элемент невозможно. Если элемент существует, его путь `items[cursor].path` записывается в переменную `marked`. Флаг `hasMarked` переводится в `true`. Это означает, что в программе теперь есть отмеченный объект. После отметки объект визуально выделяется символом `*` в функции `draw()`, если он находится в текущем отображаемом списке.

Приложение запоминает выбранный объект и может позже использовать его в функциях `copyMarked()` и `moveMarked()`.

Функция `copyMarked()`

```
void copyMarked()
```

Она копирует отмеченный ранее файл или папку в текущую рабочую директорию.

Сначала проверяется флаг `hasMarked`. Если пользователь еще не отметил объект, функция сразу завершается. Далее формируется путь назначения `dest`. Он состоит из текущей директории `currentDir` и имени отмеченного объекта `marked.filename()`. Функция проверяет, является ли отмеченный объект директорией, с помощью `fs::is_directory(marked)`. Если отмеченный объект является директорией, вызывается `fs::copy()` с параметрами `recursive` и `overwrite_existing`. Параметр `recursive` позволяет копировать папку вместе со всем содержимым, а `overwrite_existing` разрешает перезаписывать уже существующие файлы. Если отмеченный объект является файлом, вызывается `fs::copy_file()` с параметром `overwrite_existing`. После копирования вызывается `load()`, чтобы новая копия появилась в списке текущей директории.

Отмеченный объект копируется в текущую папку под тем же именем.

Функция `moveMarked()`

```
void moveMarked()
```

Она перемещает отмеченный ранее объект в текущую рабочую директорию.

Сначала проверяется `hasMarked`. Если отмеченного объекта нет, функция ничего не делает. Создается путь назначения `dest`: текущая рабочая директория плюс имя отмеченного файла или папки. Затем вызывается `fs::rename(marked, dest)`. Для пользователя это выглядит как перемещение объекта. Фактически объект получает новый путь. После успешного перемещения переменная `marked` обновляется и начинает хранить новый путь `dest`. После этого вызывается `load()`, чтобы обновить список файлов текущей директории.

Отмеченный файл или папка переносится в текущую директорию, а отметка указывает уже на новое расположение объекта.

Функция `viewFile()`

```
void viewFile()
```

Она открывает режим просмотра содержимого выбранного файла.

Сначала функция проверяет два условия: список `items` не должен быть пустым, а выбранный элемент не должен быть директорией. Если выбранный элемент является папкой, функция завершается, потому что просматривать содержимое папки как текст нельзя. Консоль очищается с помощью `system("cls")`, чтобы

вывести только содержимое файла. Файл открывается через `ifstream` в бинарном режиме `ios::binary`. Такой режим позволяет прочитать файл целиком без автоматических преобразований символов перевода строки. Содержимое файла считывается в строку `string text` с помощью итераторов `istreambuf_iterator<char>`. Этот способ позволяет получить весь файл одной строкой. На экран выводится заголовок `VIEW` и имя файла. Затем строка `text` передается в функцию `toWide()`, чтобы преобразовать байтовый текст в широкую строку. Полученная широкая строка выводится через `wcout`. В конце выводится сообщение `Press any key to return...`, после чего `_getch()` ожидает нажатия любой клавиши. После нажатия клавиши функция завершается, и основной цикл снова вызовет `draw()`, вернув пользователя к списку файлов.

Пользователь временно переходит из режима файлового менеджера в режим просмотра содержимого выбранного файла.

Функция `main()`

```
int main()
```

Она является точкой входа в программу и управляет основным циклом файлового менеджера.

В начале `main()` вызывается `_setmode(_fileno(stdout), _O_U16TEXT)`. Это переводит стандартный вывод в режим широких символов UTF-16. Аналогично настраивается `stderr`, чтобы сообщения об ошибках также могли корректно выводиться широкими символами. После настройки консоли вызывается `load()`, который загружает содержимое стартовой директории. Далее начинается бесконечный цикл `while (true)`. Внутри него программа постоянно перерисовывает экран и ожидает команду пользователя. На каждой итерации вызывается `draw()`, которая показывает текущую директорию и список элементов. Затем вызывается `key()`, которая возвращает числовой код команды. Если код равен 1 и курсор не находится на первом элементе, `cursor` уменьшается. Это перемещение вверх. Если код равен 2 и ниже есть следующий элемент, `cursor` увеличивается. Это перемещение вниз. Коды 3, 4, 5, 6, 7, 8 и 9 вызывают соответствующие функции: удаление, копирование, перемещение, просмотр файла, открытие директории, переход в родительскую папку и отметку элемента. Если код равен 10, выполняется `break`, и бесконечный цикл завершается. После выхода из цикла функция возвращает 0, что означает нормальное завершение программы.

main() связывает все остальные функции в единую программу: загрузка данных, вывод интерфейса, обработка клавиш и выполнение файловых операций.

2.5. Общая схема работы приложения

Программа определяет текущую директорию и загружает список ее содержимого. Затем в бесконечном цикле выполняются три операции: отображение интерфейса, считывание клавиши и выполнение команды.

Например, если пользователь нажимает стрелку вниз, программа меняет только значение cursor. Если пользователь нажимает Enter на директории, изменяется currentDir и заново вызывается load(). Если пользователь нажимает F3 на файле, вызывается viewFile(), которая временно заменяет список файлов текстом выбранного файла.

3. Описание реализации

3.1. Системные требования

Операционная система: Windows.

Необходимые заголовочные файлы: iostream, vector, string, filesystem, fstream, conio.h, windows.h, io.h, fcntl.h.

Для корректной работы filesystem необходимо использовать стандарт C++17 и новее, так как std::filesystem входит в стандарт начиная с C++17.

Приложение запускается в консоли и не требует установки дополнительных библиотек.

3.2. Как собрать

Для сборки в Visual Studio нужно создать консольный проект C++, добавить в проект файл FileManager1.cpp и включить стандарт языка C++17 или более новый. После этого проект можно собрать обычной командой Build.

При сборке из командной строки Visual Studio Developer Command Prompt можно использовать команду:

```
cl /EHsc /std:c++17 FileManager1.cpp
```

или

```
cl /EHsc /std:c++latest FileManager1.cpp
```

После успешной компиляции будет создан исполняемый файл. При компиляции также появляется промежуточный объектный файл с расширением .obj. Он содержит машинный код, полученный из исходного файла, и используется компоновщиком для создания итогового .exe.

3.3. Как пользоваться

После запуска программа сразу показывает содержимое директории, из которой был запущен исполняемый файл. Выбранный элемент отмечается символом >. Если элемент был предварительно отмечен пробелом, рядом с ним отображается символ *.

Стрелка вверх — переместить курсор на элемент выше.

Стрелка вниз — переместить курсор на элемент ниже.

Enter — открыть выбранную директорию.

Backspace — перейти в родительскую директорию.

Delete — удалить выбранный файл или папку.

Space — отметить выбранный файл или папку.

F1 — скопировать отмеченный объект в текущую директорию.

F2 — переместить отмеченный объект в текущую директорию.

F3 — просмотреть содержимое выбранного файла.

Esc — завершить работу программы.

Для копирования или перемещения нужно сначала выбрать нужный файл или папку, нажать Space, затем перейти в целевую директорию и нажать F1 для копирования или F2 для перемещения. Для просмотра файла нужно поставить курсор на файл и нажать F3. Возврат из режима просмотра выполняется нажатием любой клавиши.